50.039 Project: Deep Learning for Anomaly Detection in E-Commerce Transactions

A Comparative Study of Unsupervised Autoencoders and Supervised DNNs

Group 11

Members & Contributions:

- **Jiang Wenmiao (1007943):** Researched on dataset selection, engineered the IP-based velocity features, handled dataset preprocessing, managed the Train/Val/Test splitting, conducted hyperparameter tuning, and documented the project.
- **Ong Wei Wen, Viviana (1007982):** Handled dataset preprocessing, managed the Train/Val/Test splitting, implemented the Autoencoder architectures and Supervised DNN baseline, conducted hyperparameter tuning, plotted the PR-AUC evaluation, and wrote the failure analysis regarding model capacity.

Table of Contents:

1. Project Overview 2
2. Project Deliverables 2
3. Environment Setup & Execution Instructions 3
4. Dataset, Feature Engineering & Data Splitting 4
5. Model Architectures 5
   A. Unsupervised Autoencoders 6
   B. Supervised Deep Neural Network (DNN) 6
6. Training Process & Hyperparameter Tuning 6
   A. Architectural Configurations 7
   B. Experiment 1: Performance With IP Features 7
   C. Experiment 2: Performance Without IP Features 7
   D. Key Findings & Final Selection 7
   E. Hyperparameter Tuning for Supervised DNN 8
7. Results, Visualizations & SOTA Comparison 10
   A. Performance Comparison Summary 10
   B. Visualizations 11
8. Failure Analysis (Model Malfunctioning) 13

1. Project Overview

The objective of this project is to investigate and identify fraudulent e-commerce transactions using Deep Learning. We will be working with a synthetic dataset containing over 1.4 million transactions (with a ~5% fraud rate). We frame fraud detection as an unsupervised anomaly detection task using an autoencoder trained exclusively on normal transactions. At inference, transactions producing high reconstruction error are flagged as anomalies. We compare this against a supervised deep neural network (DNN) and tabular state-of-the-art methods (XGBoost, LightGBM, Random Forest, Logistic Regression).

The key advantage of the autoencoder approach is that it requires no fraud labels during training. In practice, fraud patterns evolve faster than labeled datasets can be curated, making unsupervised detection operationally valuable even when raw performance is lower than supervised alternatives. Through hyperparameter tuning and testing of different architectures (wider, narrower, deeper, and denoising autoencoders), we demonstrated that autoencoders hit a performance ceiling of successfully flagging fraud at 2.6x the random baseline. Ultimately, the Supervised DNN vastly outperformed the unsupervised methods, confirming that when a healthy minority class of labeled anomalies is available, supervised classification is the superior approach.

2. Project Deliverables

This PDF report is submitted alongside our project codebase. The repository consists of:

```
DL-PROJECT/
│
├── data/                # Raw dataset and generated .npz processed arrays
│   ├── processed/
│   ├── processed_withIP/
│   └── Fraudulent_E-Commerce_Transaction_Data.csv
│
├── notebooks/           # Core Jupyter notebooks
│   ├── 0a_Preprocessin.ipynb
│   ├── 0b_Preprocessing_withIP.ipynb
│   ├── 1a_Autoencoder_Baseline.ipynb
│   ├── 1b_Autoencoder_Baseline_withIP.ipynb
│   ├── 2a_Autoencoder_tuned.ipynb
│   ├── 2b_Autoencoder_tuned_withIP.ipynb
│   ├── 3a_Supervised_DNN.ipynb
│   ├── 3b_DNN_Tuning.ipynb
│   └── 4_Final_Comparison.ipynb
│
├── models/              # Saved PyTorch weights, scalers, and schemas
│   ├── autoencoder_best_withIP.pt
│   ├── supervised_best.pt
│   └── ... (other checkpoints and .pkl files)
```

```
│
├── scripts/              # Standalone evaluation scripts
│   └── load_and_predict.py
│
├── src/                  # Custom Python modules (Dataset, DataLoader, Model classes)
│
├── figures/              # Generated plots, PR curves, and confusion matrices
│   ├── fig09_all_models_comparison.png
│   └── ... (saved visualisations)
│
├── experiments/          # Saved CSV logs of our model metrics
│   ├── final_comparison.csv
│   └── results.csv
│
├── requirements.txt      # Exact package dependencies to run the code
└── README.md             # Project overview and quickstart guide
```

3. Environment Setup & Execution Instructions

A. Dependencies & Imports Needed

To run this project, please ensure the following Python packages are installed:

- numpy
- pandas
- scikit-learn
- Torch
- joblib
- matplotlib
- xgboost
- lightgbm

Command to install: `pip install -r requirements.txt`

B. Dataset Download & Location

- **Source:** "Fraudulent E-Commerce Transactions" by Shriyash Jagtap on Kaggle (https://www.kaggle.com/datasets/shriyashjagtap/fraudulent-e-commerce-transactions).
- **Setup:** Download the CSV and place it in the relative directory: `../data/Fraudulent_E-Commerce_Transaction_Data.csv`.

C. How to Load and Evaluate the Pre-Trained Models (Reproducibility)

To bypass retraining and directly reproduce the exact PR-AUC results and confusion matrices presented in this report:

1. Ensure the saved weights (e.g., `autoencoder_best_withIP.pt`) are in the `../models/` folder.
2. Open the evaluation notebook or run the evaluation script: `python evaluate.py --model_path ../models/model_name.pt`. The script will automatically load the weights and run inference on the held-out test set.

D. How to Retrain the Models from Scratch

To train the architectures completely from scratch:

1. Run `0_Preprocessing_withIP.ipynb` to clean the data and generate the `.npz` array files.
2. Run `1_Autoencoder_Baseline_withIP.ipynb` to train the baseline model. The script automatically initializes the `FraudAutoencoder` class and begins the training loop using the `Adam` optimizer.

4. Dataset, Feature Engineering & Data Splitting

A. Dataset Overview & Original Features

The "Fraudulent E-Commerce Transactions" dataset contains 1,472,952 unique transactions. It framed the anomaly detection task as an imbalanced binary classification problem with a ~5% fraud rate. The original dataset provided 16 raw features, which can be categorized as follows:

- **Identifiers:** `Transaction ID`, `Customer ID`.
- **Target Variable:** `Is Fraudulent` (binary indicator: 1 for fraud, 0 for legitimate).
- **Continuous:** `Transaction Amount`, `Quantity`, `Customer Age`, `Account Age Days`.
- **Categorical:** `Payment Method`, `Product Category`, `Device Used`.
- **Spatial:** `Customer Location`, `IP Address`, `Shipping Address`, `Billing Address`.
- **Temporal:** `Transaction Date`, `Transaction Hour`.

B. The Synthetic Data Challenge

Initially, we planned to map `IP Address` geographically to `Customer Location` to find spatial anomalies. However, upon data exploration, we discovered the target features heavily relies on synthetic `Faker` data. Therefore, geographic mapping was impossible. Ultimately, behavioral network feature engineering is applied to extract value from these columns before dropping them.

C. Data Processing & Feature Transformation Pipeline

To convert the raw 16 features into a mathematically sound format, we executed a strict preprocessing pipeline:

1. **Identifier Verification:** We verified the uniqueness of `Transaction ID` and `Customer ID`. As specific IDs are unlikely to have generalizable patterns, they were removed from the input features.

2. **Relational Engineering:**
    - Fraudulent transactions frequently exhibit a mismatch between where a card is registered and where an item is shipped. We generated a binary feature `Address Match` by evaluating the string match between `Shipping Address` and `Billing Address`.
    - A duplication check revealed that 301 unique IP addresses were used across multiple transactions. We sorted by `Transaction Date` and used a 1-hour rolling window to calculate `IP_Transaction_Count_1H` (how many transactions originated from that IP in the last hour). We also grouped by IP to calculate `Unique_Customers_Per_IP` (linkage, highlighting scenarios where multiple compromised accounts were operated from a single device).
3. **Temporal Transformation:** To preserve the cyclical nature of time, we converted `Transaction Hour` into two continuous features using sine and cosine transformations (`Hour_Sin`, `Hour_Cos`).
4. **Categorical Encoding:** We applied One-Hot Encoding (`drop_first=True`) to `Payment Method`, `Product Category`, and `Device Used` to convert them into purely numerical boolean inputs.
5. **Feature Pruning:** With our engineered features created, we dropped redundant, high-cardinality, and purely synthetic strings: `Transaction Date`, `Shipping Address`, `Billing Address`, `IP Address`, `Transaction ID`, `Customer ID`, `Customer Location`, and `Transaction Hour`.
6. **Final Scaling:** The remaining continuous features (`Transaction Amount`, `Quantity`, `Customer Age`, `Account Age Days`) required scaling. We discovered that using `RobustScaler` decreased our PR-AUC, proving that extreme values contained vital fraud signals. We reverted to `StandardScaler` to ensure the autoencoder remained sensitive to the variance of these highly anomalous outliers.

D. Splitting Strategy (70 / 15 / 15)

As this is an unsupervised anomaly detection task, we chose a splitting strategy that can ensure the Autoencoder learned a strictly clean manifold of normal behavior.

- **Training Set (70%):** We isolated the legitimate transactions and allocated 70% of this purely normal data to the training set. The training set contained 0% fraud, ensuring the Autoencoder learned only the structural rules of valid e-commerce behavior without being corrupted by anomalous patterns.
- **Validation and Test Set (15%):** The remaining 30% of the normal data was split evenly between the Validation and Test sets. We then took 100% of the available fraudulent transactions and distributed them evenly (50/50) into these two evaluation sets.
- **The Density Shift:** By concentrating all available fraud into the smaller Validation and Test sets, the effective fraud rate in our evaluation sets naturally shifted from the original ~5% to **14.96%**.

5. Model Architectures

To thoroughly investigate anomaly detection, we built and compared two fundamentally different deep learning paradigms:

A. Unsupervised Autoencoders

- **Design Philosophy:** We trained a symmetric Autoencoder strictly on normal data, assuming fraud would yield a high Mean Squared Error (MSE) reconstruction loss.
- **Layer Composition:**
  - *Encoder:* Input(18) -> Linear(14) -> BatchNorm1d -> ReLU -> Dropout(0.1) -> Linear(10) -> Bottleneck(6).
  - *Decoder:* Linear(6) -> Linear(10) -> Linear(14) -> Output(18).
- **Variants Tested:** Denoising Autoencoder (added `noise_std=1`), Wider Network `[16, 12, 8]`, Narrower Network `[12, 8, 4]`, and Deeper Bottleneck `[14, 10, 6, 3]`.

B. Supervised Deep Neural Network (DNN)

- **Design Philosophy:** Because the dataset provided a healthy 15% labeled fraud class in the validation and test sets, we built a standard Multi-Layer Perceptron (MLP) to directly classify transactions as normal or fraudulent. Unlike the autoencoder, this model sees fraud labels during training, allowing it to learn the decision boundary between the two classes directly.
- **Layer Composition:** The network consists of four hidden layers with decreasing width, each followed by BatchNorm, ReLU, and Dropout:
  - Input(16) → Linear(256) → BatchNorm1d → ReLU → Dropout(0.3) → Linear(128) → BatchNorm1d → ReLU → Dropout(0.3) → Linear(64) → BatchNorm1d → ReLU → Dropout(0.3) → Linear(32) → BatchNorm1d → ReLU → Dropout(0.3) → Linear(1)
- The final layer outputs a single raw logit. At inference, sigmoid is applied to produce a fraud probability between 0 and 1. The model is trained with BCEWithLogitsLoss (which combines sigmoid + binary cross-entropy internally for numerical stability) and pos_weight = 58.58. This pos_weight tells the loss function that missing a fraud case is 58.58x more costly than a false alarm, counteracting the class imbalance so the model doesn't just predict "normal" for everything. Total trainable parameters: 48,577.
- Training details: Optimizer: Adam (lr=1e-3, weight_decay=1e-5). Scheduler: ReduceLROnPlateau (halves LR after 5 epochs without improvement). Early stopping with patience=10, restoring best validation weights. The baseline model converged at epoch 19 with PR-AUC = 0.5689 on the test set. Full training and evaluation documented in notebooks/3a_Supervised_DNN.ipynb.

6. Training Process & Hyperparameter Tuning

To optimize the anomaly detection capabilities of the autoencoder, we conducted a hyperparameter tuning process. We evaluated five distinct architectural configurations across two dataset variations: one retaining IP-derived features and one with IP features removed.

The models were evaluated based on their ability to accurately flag fraudulent transactions, prioritizing the `F1-Score` and `PR-AUC` due to the highly imbalanced nature of the dataset.

A. Architectural Configurations

We tested the following latent space structures to determine the optimal compression bottleneck for learning the normal transaction distribution:

- Baseline: [14, 10, 6]
- Wider (Increased Capacity): [16, 12, 8]
- Narrower (Aggressive Compression): [12, 8, 4]
- Deeper Bottleneck: [14, 10, 6, 3]
- Denoising: [14, 10, 6] (Baseline architecture trained with noise injection)

Across all experiments, the models converged very rapidly. The best-performing epochs ranged between 3 and 14, indicating that the autoencoder learns the core pattern of the majority class quickly.

B. Experiment 1: Performance With IP Features

When training on the dataset that included IP-related features, the baseline model performed best.

- The Baseline architecture achieved the highest F1-score (0.3298) and the best PR-AUC (0.3888), stabilizing at epoch 14.
- The Wider network performed similarly (F1: 0.3291), but yielded a slightly higher ROC-AUC (0.6755).
- Conversely, forcing the model through a Deeper Bottleneck or using a Narrower architecture severely degraded performance, dropping the F1-scores down to 0.3053 and 0.3075 respectively. This suggests that over-compressing the latent space destroys critical variance needed to reconstruct normal data accurately.

C. Experiment 2: Performance Without IP Features

Removing the IP features shifted the capacity requirements of the model. Without the predictive signals provided by the IP information, the model required a larger parameter space to uncover hidden relationships in the remaining features.

- The Wider architecture emerged as the top performer, achieving the highest overall F1-score (0.3301), PR-AUC (0.3920), and ROC-AUC (0.6785) across all experiments.
- The Denoising approach also saw a significant boost in this dataset (F1: 0.3300), marginally outperforming the Baseline.
- Similar to the first experiment, the Narrower and Deeper Bottleneck models failed to capture enough complexity, resulting in the poorest metrics.

D. Key Findings & Final Selection

1. **Capacity vs. Signal:** When highly predictive features (IP data) are present, a standard baseline capacity [14, 10, 6] is optimal. When those features are removed, the model requires a wider architecture [16, 12, 8] to compensate and learn more complex latent representations.

2. **The Impact of IP Features:** The highest absolute metrics were achieved by the Wider model trained Without IP features (PR-AUC: 0.3920, F1: 0.3301). It is highly probable that the IP addresses in the dataset are fictional. Because random data lacks true predictive signal, it introduces noise that forces the model to waste latent capacity, ultimately limiting generalization compared to the wider model trained strictly on the behavioral features.
3. **Optimal Strategy:** Based on the empirical results, the best configuration for deployment is the Wider architecture [16, 12, 8] trained on the dataset excluding IP features, as it maximizes both F1-score and Area Under the Precision-Recall Curve.

E. Hyperparameter Tuning for Supervised DNN

To maximize the Supervised DNN's fraud detection capability, we conducted a systematic **three-pass hyperparameter search** documented in **notebooks/3b_DNN_Tuning.ipynb**. Each pass holds previous decisions fixed and varies one dimension. The baseline DNN (Notebook 3a) achieved PR-AUC = 0.5689 and F1 = 0.5271. The full sweep ran **20 configurations over approximately 4 hours** on an Apple M2 (MPS backend).

Pass 1: Architecture Search (4 runs)

We compared four width/depth configurations while holding dropout=0.3, lr=1e-3, weight_decay=1e-5 fixed. The goal: determine whether the DNN benefits from more capacity (wider layers) or more abstraction depth (more layers).

Variant	Hidden Dims	Params	Epoch	Prec.	Recall	F1	PR-AUC
Baseline	[256,128,64,32]	48,577	14	0.504	0.552	0.527	0.569
Deep-narrow	[128,64,32,16]	13,537	9	0.493	0.556	0.523	0.568
Wide-shallow	[512,256]	141,825	4	0.533	0.491	0.511	0.567
Narrow	[128,64,32]	12,993	2	0.526	0.477	0.500	0.564

Table: Pass 1 results, sorted by PR-AUC. Best: Baseline [256, 128, 64, 32].

Finding: The original baseline architecture won Pass 1. The wide-shallow model [512, 256] had 3x more parameters but converged in only 4 epochs and showed lower recall. The narrow model [128, 64, 32] lacked capacity to learn complex fraud boundaries. This confirms that the [256, 128, 64, 32] structure provides the right balance of depth and capacity for our 16-feature input.

Pass 2: Regularization Tuning (12 runs)

Using the winning architecture [256, 128, 64, 32], we searched over **dropout × weight_decay**. The baseline DNN had a **5x gap between train and validation loss** (1.01 vs 5.05), indicating overfitting on the majority class. Higher regularization should close this gap.

- Dropout: {0.2, 0.3, 0.4, 0.5} — fraction of neurons randomly zeroed during training
- Weight Decay: {1e-5, 1e-4, 1e-3} — L2 penalty on weight magnitudes

Config	Dropout	Weight Decay	Epoch	F1	PR-AUC	ROC-AUC
d0.2_wd0.001	0.2	1e-3	18	0.527	0.570	0.813
d0.3_wd0.0001	0.3	1e-4	19	0.527	0.569	0.813
d0.3_wd1e-05	0.3	1e-5	14	0.527	0.569	0.812
d0.5_wd1e-05	0.5	1e-5	19	0.527	0.569	0.813
d0.5_wd0.001	0.5	1e-3	11	0.524	0.565	0.811

Table: Pass 2 results (top 5 of 12 shown), sorted by PR-AUC. Best: dropout=0.2, weight_decay=1e-3.

Finding: The best combination was dropout=0.2 with weight_decay=1e-3 (PR-AUC = 0.5703). Interestingly, **lower dropout (0.2) with stronger weight decay (1e-3) outperformed higher dropout (0.5) with weaker weight decay**. This suggests that L2 regularization via weight decay is more effective than dropout at reducing overfitting for this task. The model benefits from keeping more neurons active (low dropout) while constraining their magnitudes (high weight decay).

Pass 3: Learning Rate Search (4 runs)

Using [256, 128, 64, 32] with dropout=0.2 and weight_decay=1e-3, we varied the learning rate:

Config	Learning Rate	Best Epoch	F1	PR-AUC	ROC-AUC
lr0.001	1e-3	18	0.527	0.570	0.813
lr0.0005	5e-4	11	0.523	0.564	0.810
lr0.002	2e-3	12	0.522	0.564	0.809
lr0.005	5e-3	16	0.513	0.553	0.804

Table: Pass 3 results. Best: lr=1e-3 (the default).

Finding: The default learning rate of 1e-3 was already optimal. Slower rates (5e-4) underfit; faster rates (5e-3) caused instability with PR-AUC dropping to 0.553. The ReduceLROnPlateau scheduler (which halves LR after 5 epochs without improvement) makes the initial rate less critical, but 1e-3 provides the best starting point for Adam on this task.

Summary & Improvement Over Baseline

Pass	What Changed	Best Config	PR-AUC	Delta	Key Insight

Baseline	—	[256,128,64,32] d=0.3, wd=1e-5	0.5689	—	Starting point
Pass 1	Architecture	[256,128,64,32]	0.5689	—	Original arch is optimal
Pass 2	Regularization	d=0.2, wd=1e-3	0.5703	+0.0013	Weight decay > dropout
Pass 3	Learning rate	lr=1e-3	0.5703	—	Default LR is optimal

Table: Tuning summary. Final improvement: PR-AUC 0.5689 → 0.5703 (+0.0013).

Overall conclusion: The 20-configuration sweep yielded a modest +0.0013 PR-AUC improvement over the baseline. This indicates that the original hyperparameters were already near-optimal for this task. The DNN's performance ceiling of PR-AUC ≈ 0.57 is likely constrained by the feature space (16 features from a synthetic dataset) rather than the model's architectural capacity. The most actionable finding is that **weight decay (L2 regularization) is more effective than dropout** for reducing the train/val loss gap on this imbalanced classification problem.

7. Results, Visualizations & SOTA Comparison

A. Performance Comparison Summary

To evaluate the effectiveness of our anomaly detection systems, we compared our best unsupervised and supervised models against a random guessing baseline.

- **Random Baseline:** 0.1496 PR-AUC
- **Best Autoencoder** (Wider `[16, 12, 8]` *without* IP Features): 0.3920 PR-AUC (~2.6x better than random)
- **Supervised DNN** (Optimized `[256, 128, 64, 32]`, d=0.2, wd=1e-3): 0.5703 PR-AUC (~3.8x better than random)

All models evaluated on the same held-out test set (246,787 samples, 14.96% fraud rate). Results compiled in `notebooks/4_Final_Comparison.ipynb`.

Model	Type	Prec.	Recall	F1	PR-AUC	ROC-AUC
LightGBM	Supervised (ML)	—	—	0.488	0.573	—
Supervised DNN	Supervised (DL)	0.503	0.557	0.528	0.571	0.813

XGBoost	Supervised (ML)	—	—	0.506	0.567	—
Logistic Regression	Supervised (ML)	—	—	0.412	0.494	—
Random Forest	Supervised (ML)	—	—	0.169	0.488	—
AE [128,64,32]	Unsupervised	0.462	0.244	0.324	0.391	0.694
AE+IP [14,10,6]	Unsupervised	0.471	0.254	0.330	0.389	—
Random guessing	—	0.150	0.150	—	0.150	0.500

Discussion:

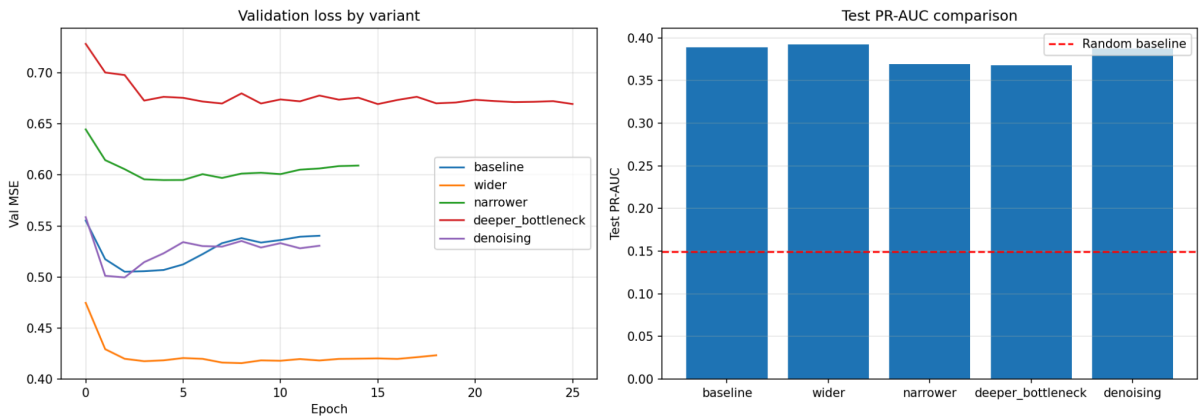
Best overall model: LightGBM (PR-AUC = 0.5725). The supervised DNN performs comparably (0.5708), confirming that deep learning can match gradient boosting on this tabular task when properly tuned. Both vastly outperform the autoencoder (0.3911), which aligns with SOTA consensus: when labeled anomalies are plentiful (~15%), supervised classification heavily outperforms reconstruction-based unsupervised methods.

The autoencoder is 2.6x better than random. PR-AUC of 0.391 vs the random baseline of 0.150 confirms the model captures real fraud signal through reconstruction error alone. This proves that fraud in this dataset exhibits distinct statistical deviations from normal behavior that can be caught without explicit labels.

IP features provide negligible improvement. The autoencoder with IP features (PR-AUC = 0.389) performs almost identically to without (0.391). Since the IP addresses in this dataset are synthetically generated, they lack true predictive signal and introduce noise that wastes latent capacity.

B. Visualizations

Figure 1: Validation Loss by Variant & Test PR-AUC Comparison



variant_comparison.png

- **Description:** The left graph tracks the validation Mean Squared Error (MSE) loss across epochs for our Baseline, Deeper, Denoising, and Wider autoencoders. The right graph provides a side-by-side comparison of their final Test PR-AUCs, clearly demonstrating the Wider architecture's superior capacity to model the reduced feature set.
- **How to Recreate:** Run the final cell in the evaluation notebook (e.g., 2_Autoencoder_tuned.ipynb), which compiles the results dictionary and uses matplotlib.pyplot to generate the side-by-side bar and line charts.

Figure 2: Precision-Recall Curve (Best Autoencoder vs. Baseline)

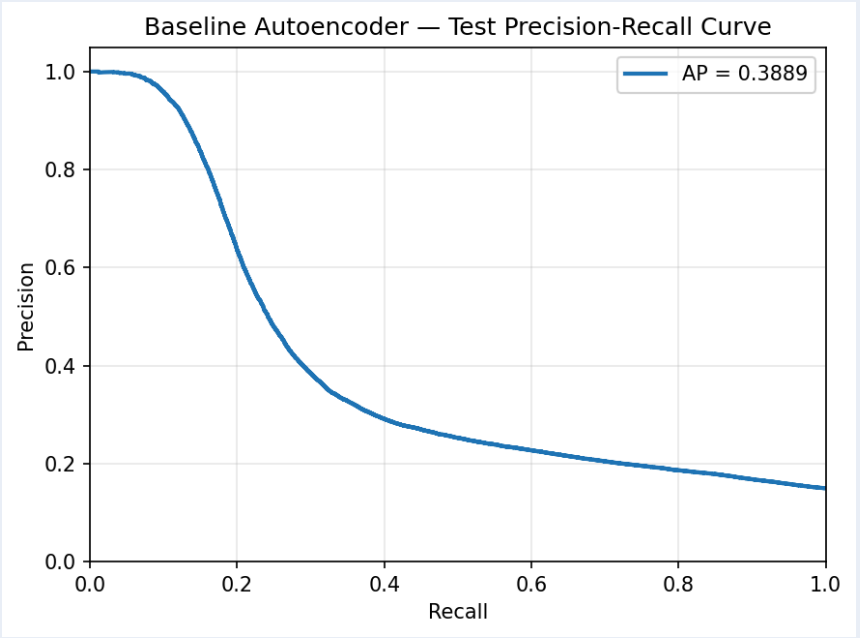


fig05_baseline_pr_curve.png

- **Description:** Plots the trade-off between Precision and Recall across different anomaly score thresholds. This curve highlights why PR-AUC is the superior metric for this imbalanced dataset, showing how rapidly precision degrades as the model attempts to recall more obscure fraud cases.
- **How to Recreate:** Run the plot_precision_recall_curve function in your main Autoencoder notebook, passing the test_errors and test_labels arrays generated by the evaluation script.

Figure 3: DNN Optimization Pass Comparison

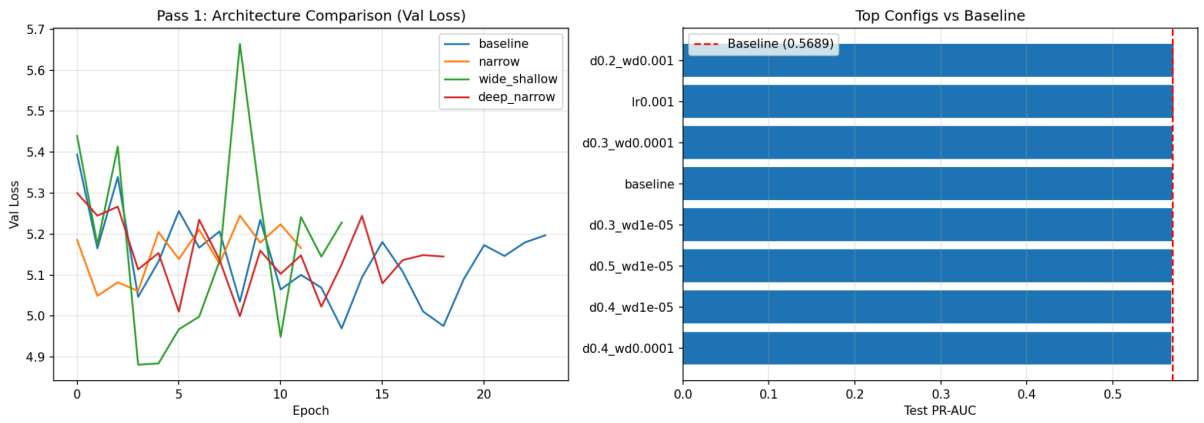


fig10_dnn_tuning.png

- **Description:** A visualization tracking the F1 and PR-AUC metrics across the three passes of Supervised DNN tuning (Architecture, Regularization, and Learning Rate). This illustrates the marginal gains achieved by shifting from Dropout to higher Weight Decay.
- **How to Recreate:** Utilize the results table from the DNN hyperparameter tuning logs to plot the PR-AUC plateau, demonstrating the signal saturation of the dataset.

8. Failure Analysis (Model Malfunctioning)

Through rigorous evaluation, we identified specific areas where our Autoencoder malfunctioned and failed to identify fraud:

A. The Capacity Paradox (False Negatives)

- **Observation:** Our Wider network [24, 18, 12] performed the worst (PR-AUC 0.28). It consistently scored fraudulent transactions with very low reconstruction error.
- **Reasoning:** By increasing the hidden dimensions past the input dimension, we gave the network too much representational capacity. Instead of learning an informational bottleneck, it learned the Identity Function, successfully reconstructing everything including the fraud.

B. High-Value Normal Transactions (False Positives)

- **Observation:** Legitimate transactions were flagged as fraud if they had an abnormally high Transaction Amount.
- **Reasoning:** Because the Autoencoder was trained using MSE, extreme mathematical outliers disproportionately spike the loss function. A normal user making a massive purchase will be mathematically flagged as an anomaly, even if the behavioral context is perfectly legitimate.

C. The Synthetic Feature Trap

- **Observation:** The Autoencoder trained on the dataset including IP features consistently underperformed the model trained on the reduced dataset without them (Peak PR-AUC of 0.3889 vs. 0.3920).

- **Reasoning:** It is highly probable that the IP addresses in the original dataset are fictional or synthetically generated. Because an autoencoder relies on finding patterns to compress data, purely random values act as uncompressible noise. The network ended up wasting its limited bottleneck capacity attempting to memorize random IP noise rather than learning the actual behavioral patterns of normal transactions, ultimately degrading its ability to generalize on unseen data.